2.2 Pipeline Algorithms and Data Structures (System Design)

Project: E-Learning Web Pages for Computer Science
Student: Zain Chohan (32001784)
Supervisor: Richard Mitchell

Overview

This section describes the **core algorithms and data structures** used to simulate the five-stage CPU pipeline (Fetch, Decode, Execute, Memory, Write-back).
The design ensures accurate instruction flow while supporting interactivity, hazard detection, and visualisation in a web environment.

The simulator follows an **event-driven architecture** implemented in JavaScript, using the **Model–View–Controller (MVC)** pattern:

- **Model:** Represents the CPU state (instructions, registers, memory, pipeline stages).
- **Controller:** Executes the pipeline simulation logic and manages user actions.
- **View:** Visualises instruction flow through the pipeline stages in real time.

Pipeline Stages

The pipeline consists of the **five classical stages**, each implemented as a modular object with its own state and transition rules:

Stage	Abbreviation	Function
Instruction Fetch	IF	Retrieves the next instruction from memory using the Program Counter (PC).
Instruction Decode / Register Read	ID	Decodes the opcode, identifies operands, and reads register values.
Execute / ALU Operation	EX	Performs arithmetic, logic, or address calculation operations.

Memory Access	MEM	Reads from or writes to data memory as required.
Write Back	WB	Updates destination registers with the results of computation or memory reads.

Each stage operates concurrently, advancing one instruction per clock cycle when no hazard is present.

Core Data Structures

To manage the pipeline efficiently, several JavaScript data structures are defined.

1. Instruction Object

Each instruction is represented as an object with fields describing its operation and pipeline state:

```
{
  id: 1,
  opcode: "ADD",
  rd: "R1",
  rs: "R2",
  rt: "R3",
  stage: "IF",
  cycleEntered: 4,
  dependencies: ["R2", "R3"],
  completed: false
}
```

Purpose:

- Stores the current pipeline stage for visualisation.
- Tracks dependencies for hazard detection.
- Provides metadata for performance metrics (cycle count, stalls, etc.).

---

2. Pipeline Register Buffers

Each stage communicates via **inter-stage pipeline registers**, represented as objects such as:

```
IF_ID = { instruction: null };
ID_EX = { instruction: null };
EX_MEM = { instruction: null };
MEM_WB = { instruction: null };
```

These buffers store instructions temporarily between stages and are updated every cycle, imitating real hardware register latching.

---

3. CPU State Objects

The simulator maintains three key shared states:

```
Registers = { R0: 0, R1: 5, R2: 10, ... };
Memory = [0, 4, 8, 12, 16, ...];
ProgramCounter = 0;
```

Purpose:

- Tracks all register and memory values in real time.
- Enables hazard detection logic to verify read-after-write (RAW) conflicts.
- Provides data for the on-screen register/memory viewer.

Pipeline Control Algorithm

The simulation progresses in discrete **clock cycles**, managed by a main loop triggered by the **step**, **play**, or **pause** controls.

Main Simulation Loop (Pseudo-code)

```
for each clock cycle:
    1. Write Back (WB) – update destination registers.
    2. Memory (MEM) – perform load/store operations.
    3. Execute (EX) – perform ALU operations.
    4. Decode (ID) – read registers, check dependencies.
    5. Fetch (IF) – fetch next instruction.
```

```
6. Update pipeline registers (shift stages forward).
7. Detect and resolve hazards.
```

Explanation:

- Stages execute in reverse order each cycle to prevent overwriting of intermediate results.
- After each cycle, the system updates the Canvas visualisation and performance counters (cycle count, CPI).

Hazard Detection and Resolution Algorithms

1. Data Hazard Detection

Checks for **RAW (Read After Write)** dependencies:

```
if (EX.instruction.rd === ID.instruction.rs ||
    EX.instruction.rd === ID.instruction.rt)
        stallPipeline();
```

2. Control Hazard Handling

When a **branch** instruction is detected in the EX stage:

- Predict **not taken** by default.
- If prediction is wrong → **flush** IF and ID pipeline registers.
- Increment branch misprediction counter.

3. Structural Hazards

Prevent concurrent use of shared resources (e.g., memory).
Detected by stage occupancy checks before advancement.

4. Forwarding (Bypassing)

If enabled, data from later stages (EX/MEM) is forwarded to earlier ones to reduce stalls:

```
if (EX.rd === ID.rs) ID.rsValue = EX.result;
```

Performance Metrics Calculation

Performance data is calculated in real time using counters:

```
CPI = total_cycles / instructions_completed;
stallCount++;
branchAccuracy = correctPredictions / totalBranches * 100;
```

Displayed dynamically using Chart.js or Recharts visualisations.

Visualisation Synchronisation Algorithm

After each simulation cycle:

1. Update internal pipeline state (Model).
2. Redraw Canvas/SVG pipeline view (View).
3. Synchronise UI controls (Controller).

Animations use `requestAnimationFrame()` for smooth transitions at ~60 FPS.

Data Flow Diagram (Conceptual)

```
Instruction Input → IF → ID → EX → MEM → WB → Output Metrics
                              ↓
                        Hazard Detection
                              ↓
                      Forwarding / Stalling
```

This flow demonstrates the interaction between stages, hazard logic, and performance tracking.

Summary

This system design defines the **core computational logic and data structures** for simulating a five-stage CPU pipeline in a browser.

Using **object-based state management, hazard detection algorithms, and cycle-accurate control flow**, the simulator achieves both **realistic behaviour** and **educational clarity**.

The modular design ensures easy maintenance, extensibility (e.g., multi-cycle operations or superscalar pipelines), and compatibility with interactive visualisation features.

Pipeline Architecture Diagram